



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 14

Sistema de Cruce Peatonal

Carrera:

Ingeniería Mecatrónica – Cuarto Semestre

Alumno:

Saúl Martínez Almazán

Clave:

Docente:

Ing. Luis Jesús Padrón Padrón

Fecha:

24 de Abril de 2026

Índice

1. Introducción	2
1.1. Objetivo	2
1.2. Trabajo Previo	2
1.2.1. Máquinas de Estado Finitas (FSM)	2
1.2.2. Multiplexado de Displays de 7 Segmentos	3
1.2.3. Debounce por Software	3
2. Desarrollo	3
2.1. Descripción General del Sistema	3
2.2. Máquina de Estados	4
2.3. Modificaciones Implementadas Respecto al Diseño Base	4
2.4. Diagramas de Conexión	6
2.5. Tiempos del Sistema	6
2.6. Proceso de Desarrollo	7
2.7. Errores Comunes y Soluciones	8
3. Conclusión	8
Anexos	9
Evidencia Fotográfica	17

1. Introducción

En la presente práctica se diseña e implementa un **sistema de cruce peatonal** controlado por una **Raspberry Pi Pico W** programada con el Pico SDK en lenguaje C. El sistema emula el funcionamiento real de una intersección semafórica coordinada, integrando dos semáforos vehiculares, dos semáforos peatonales con indicador de cuenta regresiva y señalización sonora.

La relevancia industrial de este tipo de sistemas reside en que los controladores de tráfico modernos son sistemas embebidos de tiempo real que deben garantizar **seguridad, coordinación y respuesta a eventos externos** (como la presión de un botón peatonal) sin comprometer la integridad del flujo vehicular. Los mismos principios de máquinas de estado, temporización precisa y manejo de entradas digitales con debounce presentes en esta práctica se encuentran en controladores viales industriales, automatización de procesos y sistemas de seguridad.

El sistema implementado opera bajo una **máquina de estados finita (FSM)** con seis estados bien definidos, garantizando que en todo momento los semáforos vehiculares estén coordinados de forma mutuamente excluyente. La principal modificación respecto al diseño base consiste en la reducción del tiempo de verde vehicular cuando se detecta una solicitud de cruce peatonal pendiente, lo que minimiza la espera del peatón sin comprometer la seguridad vial.

1.1. Objetivo

Diseñar e implementar en la Raspberry Pi Pico W un sistema de cruce peatonal síncrono que coordine dos semáforos vehiculares y dos semáforos peatonales mediante una máquina de estados, integrando botones de solicitud de cruce, display de 7 segmentos multiplexado para cuenta regresiva, buzzer de señalización sonora y lógica de aceleración del ciclo vehicular ante solicitudes peatonales.

1.2. Trabajo Previo

1.2.1. Máquinas de Estado Finitas (FSM)

Una **máquina de estados finita** es un modelo computacional que describe el comportamiento de un sistema mediante un conjunto finito de estados, transiciones entre ellos y acciones asociadas. En sistemas embebidos de control, la FSM es la herramienta preferida para implementar lógica de control secuencial porque:

- Hace explícito el comportamiento del sistema en cada situación posible.
- Facilita la verificación formal de la correctitud del control.
- Es directamente implementable con estructuras **switch-case** en C.
- Permite agregar nuevos estados sin reestructurar la lógica existente.

1.2.2. Multiplexado de Displays de 7 Segmentos

El **multiplexado** es la técnica que permite controlar múltiples displays de 7 segmentos con un número reducido de pines. En lugar de dedicar 7 pines a cada display, todos los displays comparten los mismos pines de segmento y se habilitan de forma alternada a alta frecuencia. El ojo humano, al no poder percibir el parpadeo por encima de aproximadamente 50 Hz, integra la imagen y percibe ambos displays como encendidos simultáneamente.

En esta práctica se utilizan **dos displays de cátodo común** multiplexados. Los pines de segmento (A–G) son compartidos, mientras que los pines COM_DISP1 y COM_DISP2 seleccionan cuál display está activo en cada ciclo. La función `mux_tick()` se invoca frecuentemente dentro de los retardos para mantener la imagen estable.

1.2.3. Debounce por Software

Los pulsadores mecánicos producen múltiples transiciones espurias al presionarse, fenómeno conocido como *rebote* (*bounce*). El debounce por software consiste en ignorar las transiciones durante un período corto (típicamente 10–50 ms) después de la primera detección, confirmando que el estado se mantiene estable antes de registrar la pulsación.

2. Desarrollo

2.1. Descripción General del Sistema

El sistema se compone de los siguientes elementos físicos conectados a la Raspberry Pi Pico W:

- **Semáforo vehicular 1 (CAR1):** LEDs rojo (GP0), amarillo (GP1) y verde (GP2).
- **Semáforo vehicular 2 (CAR2):** LEDs rojo (GP3), amarillo (GP4) y verde (GP5).
- **Semáforo peatonal 1 (PEA1):** LEDs rojo (GP6) y verde (GP7).
- **Semáforo peatonal 2 (PEA2):** LEDs rojo (GP8) y verde (GP9).
- **Buzzer:** GP10. Emite señalización sonora durante el cruce peatonal.
- **Botón peatonal 1 (PUSH1):** GP11, con pull-up interno.
- **Botón peatonal 2 (PUSH2):** GP12, con pull-up interno.
- **Display 1 (DISP1):** Cuenta regresiva del semáforo peatonal 1. Cátodo común habilitado por COM_DISP1 (GP26).
- **Display 2 (DISP2):** Cuenta regresiva del semáforo peatonal 2. Cátodo común habilitado por COM_DISP2 (GP27).
- **Segmentos compartidos:** SEG_A (GP17), SEG_B (GP16), SEG_C (GP20), SEG_D (GP21), SEG_E (GP22), SEG_F (GP18), SEG_G (GP19).

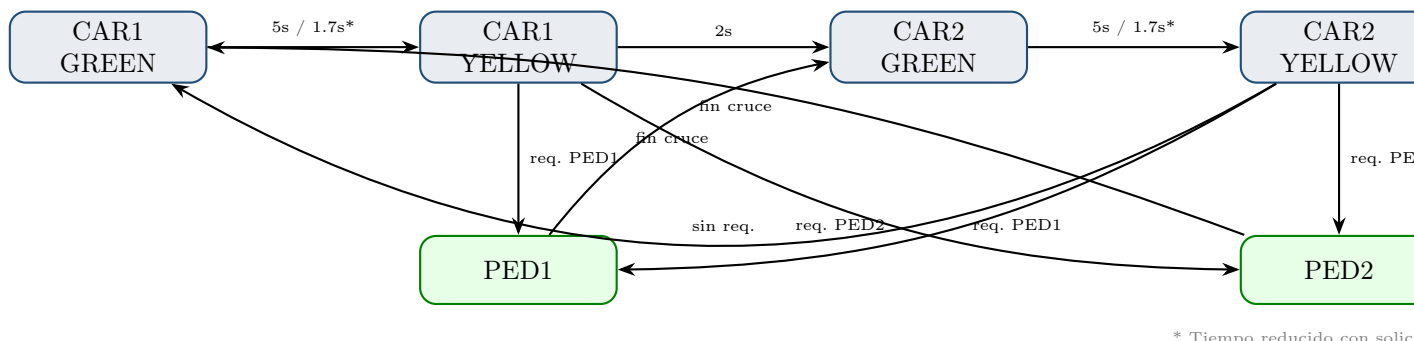
2.2. Máquina de Estados

El sistema implementa una FSM con seis estados:

Cuadro 1: Estados de la máquina de estados del sistema de cruce peatonal

Estado	Semáforos activos	Transición siguiente
STATE_CAR1_GREEN	CAR1=Verde, CAR2=Rojo	→ CAR1_YELLOW
STATE_CAR1_YELLOW	CAR1=Amarillo, CAR2=Rojo	→ PED1, PED2 o CAR2_GREEN
STATE_CAR2_GREEN	CAR1=Rojo, CAR2=Verde	→ CAR2_YELLOW
STATE_CAR2_YELLOW	CAR1=Rojo, CAR2=Amarillo	→ PED1, PED2 o CAR1_GREEN
STATE_PED1	CAR1=Rojo, CAR2=Verde	Cruce peatonal 1, → CAR2_GREEN
STATE_PED2	CAR1=Verde, CAR2=Rojo	Cruce peatonal 2, → CAR1_GREEN

El diagrama de transiciones de la FSM se presenta a continuación:



* Tiempo reducido con solicitud

Figura 1: Diagrama de transiciones de la FSM del sistema de cruce peatonal. El asterisco indica la reducción de tiempo de verde vehicular ante solicitudes peatonales pendientes.

2.3. Modificaciones Implementadas Respecto al Diseño Base

Se realizaron las siguientes adaptaciones al diseño original solicitado en la práctica:

1. **Aceleración del ciclo vehicular ante solicitud peatonal:** Cuando se detecta una solicitud peatonal activa (`ped1_request` o `ped2_request`), el tiempo de verde vehicular se reduce de `GREEN_NORMAL` (5 000 ms) a `GREEN_FAST` (1 666 ms). Esto permite que el peatón cruce antes sin tener que esperar el ciclo completo del semáforo.

2. **Un display por semáforo peatonal:** Dado que en la implementación física se dispone de dos displays de 7 segmentos, se asignó un display a cada semáforo peatonal. El display 1 muestra la cuenta regresiva del cruce peatonal 1 y el display 2 la del cruce peatonal 2. Cuando el semáforo peatonal correspondiente está en rojo, su display muestra el código 0x00 (apagado). La cuenta regresiva va de 9 a 0.
3. **Multiplexado continuo en retardos:** La función `smart_delay()` llama a `mux_tick()` y `check_buttons()` dentro del bucle de espera, garantizando que los displays permanezcan encendidos con aspecto estable durante toda la ejecución, incluyendo las fases de amarillo y verde vehicular.
4. **Buzzer de señalización sonora:** Se agregó un buzzer activo en GP10 que emite una ráfaga corta al inicio del cruce peatonal y pitidos breves durante los últimos 3 segundos de la cuenta regresiva, replicando el comportamiento de los sistemas peatonales accesibles reales.

2.4. Diagramas de Conexión

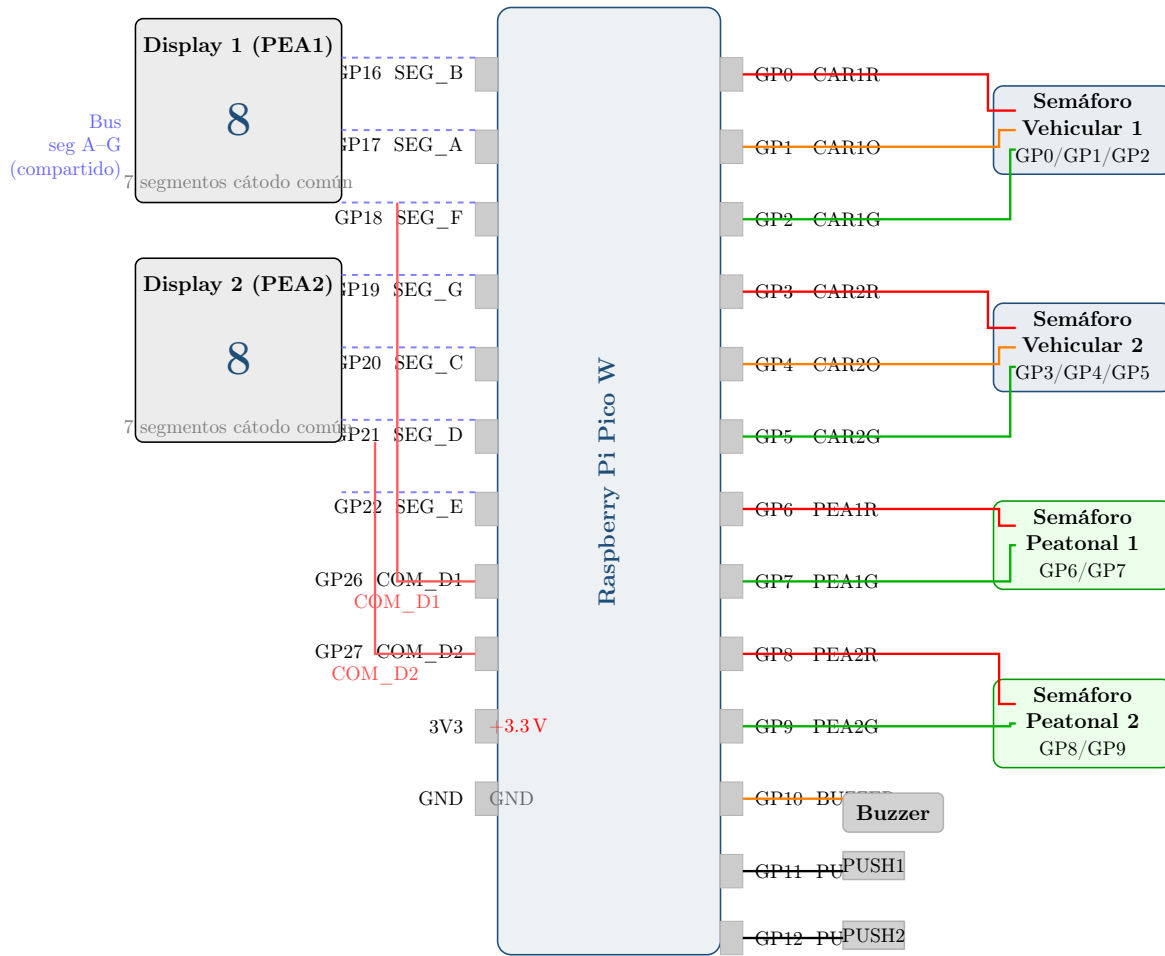


Figura 2: Diagrama esquemático del sistema de cruce peatonal. Los semáforos vehiculares y peatonales se conectan a los GPIOs con resistencias de $220\ \Omega$ en serie (omitidas por claridad). Los segmentos A–G de ambos displays de 7 segmentos son compartidos; los pines COM_D1 y COM_D2 seleccionan el display activo mediante multiplexado.

2.5. Tiempos del Sistema

Cuadro 2: Tiempos configurados en el sistema de cruce peatonal

Parámetro	Constante	Valor
Verde vehicular (sin solicitud)	GREEN_NORMAL	5 000 ms
Verde vehicular (con solicitud)	GREEN_FAST	1 666 ms
Amarillo vehicular	YELLOW_TIME	2 000 ms
Cruce peatonal	(fijo)	10 s (cuenta 9→0)

2.6. Proceso de Desarrollo

1. **Definición de pines y tipos:** Se definieron macros para todos los pines de salida (semáforos, buzzer, displays) y entradas (botones), y se declaró el `enum system_state_t` con los seis estados de la FSM.
2. **Inicialización:** La función `init_all()` itera sobre un arreglo de pines de salida para inicializarlos con `gpio_set_dir(pin, GPIO_OUT)`, y configura los botones con pull-up interno mediante `gpio_pull_up(pin)`. Los displays se deshabilitan inicialmente poniendo los pines COM en alto (cátodo común inactivo).
3. **Multiplexado de displays:** La función `mux_tick()` alterna entre los dos displays en cada llamada, apagando ambas unidades primero, escribiendo el patrón del segmento correspondiente y habilitando solo uno de los pines COM. Se llama en cada iteración de `smart_delay()` y `beep()` para mantener la imagen estable.
4. **Detección de botones con debounce:** `check_buttons()` verifica el nivel bajo del pin, espera 20 ms y confirma el estado antes de registrar la solicitud. Mientras el botón permanece presionado, sigue llamando a `mux_tick()` para no interrumpir el display.
5. **Retardo inteligente:** `smart_delay(ms)` utiliza `make_timeout_time_ms()` y `time_reached()` para implementar un retardo no bloqueante que llama a `mux_tick()` y `check_buttons()` en cada iteración.
6. **Secuencia de cruce peatonal:** La función `pedestrian_crossing()` activa el LED verde peatonal, emite un beep inicial, ejecuta una cuenta de 9 a 0 actualizando el display correspondiente, y emite pitidos adicionales en los últimos 3 segundos. Al finalizar, apaga el verde y activa el rojo peatonal.
7. **Lógica de la FSM:** `update_system()` implementa el `switch-case` sobre `current_state`, aplicando las salidas de cada estado y calculando el estado siguiente según las solicitudes peatonales activas.
8. **Compilación:** Se utilizó el Pico SDK con CMake, habilitando únicamente la biblioteca `hardware_timer` además de `pico_stdlib`.

2.7. Errores Comunes y Soluciones

Cuadro 3: Errores encontrados y soluciones aplicadas durante el desarrollo

Error Encontrado	Solución Aplicada
Los displays parpadean visiblemente durante los estados vehiculares.	Se integró <code>mux_tick()</code> dentro de <code>smart_delay()</code> , de modo que el multiplexado se ejecuta en cada iteración del bucle de espera y no solo al inicio o fin de cada estado.
El botón registra múltiples pulsaciones por un solo press (rebote).	Se implementó debounce por software: tras detectar el flanco descendente se espera 20 ms con <code>sleep_ms(20)</code> y se verifica nuevamente el estado antes de registrar la solicitud.
Typo en el pin PYA2G (debería ser PEA2G).	Se identificó el error en la definición del macro y se corrigió en todas las referencias del código, incluyendo la tabla de inicialización en <code>init_all()</code> .
Al iniciar el sistema el display muestra segmentos aleatorios.	Se forzó el estado inicial de los pines COM a alto (displays apagados) y se inicializaron <code>disp1_val</code> y <code>disp2_val</code> a 10 (código de display apagado) en <code>init_all()</code> antes del primer ciclo principal.
El cruce peatonal 2 activa los pines incorrectos.	Se verificó que <code>STATE_PED2</code> pase correctamente PEA2R y PEA2G (corregido el typo original) junto con <code>&disp2_val</code> a la función <code>pedestrian_crossing()</code> .

3. Conclusión

Esta práctica permitió consolidar el diseño de sistemas de control embebido basados en **máquinas de estados finitas** aplicadas a un escenario de ingeniería real: la coordinación de semáforos en un cruce peatonal. La implementación en lenguaje C sobre la Raspberry Pi Pico W con el Pico SDK demostró la viabilidad de gestionar múltiples periféricos (LEDs, displays, buzzer, botones) con un único hilo de ejecución mediante el patrón de *retardo no bloqueante*.

La técnica de **multiplexado de displays** resultó particularmente relevante: al integrar `mux_tick()` dentro de todos los bucles de espera se obtuvo una imagen estable sin necesidad de interrupciones o temporizadores de hardware adicionales, demostrando cómo la cooperación entre funciones de software puede suplir la ausencia de recursos

de hardware dedicados.

La modificación de **aceleración del ciclo vehicular** ante solicitudes peatonales ilustra un compromiso de diseño real: reducir la espera del peatón sin eliminar por completo el tiempo de verde vehicular, lo que sería inseguro. Este tipo de decisiones de compromiso entre fluidez y seguridad es habitual en sistemas de control vial profesionales.

Posibles mejoras:

- **Manejo de solicitudes simultáneas:** Implementar una cola de solicitudes para atender ambos botones en el mismo ciclo cuando se presionan de forma concurrente, en lugar de priorizar siempre PED1 sobre PED2.
- **Multiplexado por interrupción de timer:** Mover `mux_tick()` a un callback de `hardware_timer` para garantizar una frecuencia de refresco constante independiente del tiempo de ejecución de cada estado.
- **Display LCD:** Agregar una pantalla LCD para mostrar el estado actual del sistema y el tiempo restante del semáforo vehicular activo.
- **Control por PWM:** Implementar parpadeo del LED verde peatonal en los últimos 3 segundos mediante PWM en lugar de encendido fijo, mejorando la visibilidad de la cuenta regresiva.
- **Modo nocturno:** Agregar un modo de parpadeo amarillo en ambos semáforos vehiculares activable por horario o sensor de luminosidad, emulando el comportamiento de los semáforos reales fuera de horario.

Anexos

Anexo A – Código Fuente en C: Practica14.c

```
1 #include <stdio.h>
2 #include "pico/stdlib.h"
3 #include "hardware/timer.h"
4
5 //===== VEHICULARES =====
6 #define CAR1R 0
7 #define CAR1O 1
8 #define CAR1G 2
9
10 #define CAR2R 3
11 #define CAR2O 4
12 #define CAR2G 5
13
14 //===== PEATONALES =====
15 #define PEA1R 6
```

```
16 #define PEA1G 7
17
18 #define PEA2R 8
19 #define PEA2G 9
20
21 //===== BUZZER =====
22 #define BUZZER 10
23
24 //===== BOTONES =====
25 #define PUSH1 11
26 #define PUSH2 12
27
28 //===== DISPLAY =====
29 #define SEG_B 16
30 #define SEG_A 17
31 #define SEG_F 18
32 #define SEG_G 19
33 #define SEG_C 20
34 #define SEG_D 21
35 #define SEG_E 22
36
37 #define COM_DISP1 26
38 #define COM_DISP2 27
39
40 //===== TIEMPOS =====
41 #define GREEN_NORMAL 5000
42 #define GREEN_FAST 1666
43 #define YELLOW_TIME 2000
44
45 //=====
46 typedef enum{
47     STATE_CAR1_GREEN,
48     STATE_CAR1_YELLOW,
49     STATE_CAR2_GREEN,
50     STATE_CAR2_YELLOW,
51     STATE_PED1,
52     STATE_PED2
53 } system_state_t;
54
55 volatile system_state_t current_state = STATE_CAR1_GREEN;
56
57 volatile bool ped1_request = false;
58 volatile bool ped2_request = false;
59
60 volatile int disp1_val = 10;
61 volatile int disp2_val = 10;
62
63 //=====
```

```
64 // DISPLAY CATODO COMUN
65 const uint8_t seg7[11] = {
66     0x3F, //0
67     0x06, //1
68     0x5B, //2
69     0x4F, //3
70     0x66, //4
71     0x6D, //5
72     0x7D, //6
73     0x07, //7
74     0x7F, //8
75     0x6F, //9
76     0x00  //apagado
77 };
78
79 //=====
80 void init_output(uint pin){
81     gpio_init(pin);
82     gpio_set_dir(pin, GPIO_OUT);
83     gpio_put(pin,0);
84 }
85
86 void init_input(uint pin){
87     gpio_init(pin);
88     gpio_set_dir(pin, GPIO_IN);
89     gpio_pull_up(pin);
90 }
91
92 //=====
93 void set_car1(bool r,bool y,bool g){
94     gpio_put(CAR1R,r);
95     gpio_put(CAR10,y);
96     gpio_put(CAR1G,g);
97 }
98
99 void set_car2(bool r,bool y,bool g){
100     gpio_put(CAR2R,r);
101     gpio_put(CAR20,y);
102     gpio_put(CAR2G,g);
103 }
104
105 //=====
106 void write_segments(uint8_t pattern){
107     gpio_put(SEG_A, (pattern >> 0) & 1);
108     gpio_put(SEG_B, (pattern >> 1) & 1);
109     gpio_put(SEG_C, (pattern >> 2) & 1);
110     gpio_put(SEG_D, (pattern >> 3) & 1);
111     gpio_put(SEG_E, (pattern >> 4) & 1);
```

```
112     gpio_put(SEG_F, (pattern >> 5) & 1);
113     gpio_put(SEG_G, (pattern >> 6) & 1);
114 }
115
116 //=====
117 void mux_tick(){
118     static bool current = false;
119
120     gpio_put(COM_DISP1,1);
121     gpio_put(COM_DISP2,1);
122
123     if(!current){
124         write_segments(seg7[disp1_val]);
125         gpio_put(COM_DISP1,0);
126     }
127     else{
128         write_segments(seg7[disp2_val]);
129         gpio_put(COM_DISP2,0);
130     }
131
132     current = !current;
133 }
134
135 //=====
136 void check_buttons(){
137     if(!gpio_get(PUSH1)){
138         sleep_ms(20);
139         if(!gpio_get(PUSH1)){
140             ped1_request = true;
141             while(!gpio_get(PUSH1)){
142                 mux_tick();
143             }
144         }
145     }
146
147     if(!gpio_get(PUSH2)){
148         sleep_ms(20);
149         if(!gpio_get(PUSH2)){
150             ped2_request = true;
151             while(!gpio_get(PUSH2)){
152                 mux_tick();
153             }
154         }
155     }
156 }
157
158 //=====
159 void smart_delay(uint32_t ms){
```

```
160     absolute_time_t end = make_timeout_time_ms(ms);
161     while(!time_reached(end)){
162         mux_tick();
163         check_buttons();
164     }
165 }
166
167 //=====
168 void beep(uint32_t ms){
169     absolute_time_t end = make_timeout_time_ms(ms);
170     while(!time_reached(end)){
171         gpio_put(BUZZER,1);
172         sleep_us(500);
173         gpio_put(BUZZER,0);
174         sleep_us(500);
175         mux_tick();
176     }
177 }
178
179 //=====
180 void pedestrian_crossing(
181     uint pedR,
182     uint pedG,
183     volatile int *display_val
184 ){
185     gpio_put(pedR,0);
186     gpio_put(pedG,1);
187
188     beep(300);
189
190     for(int i=9;i>=0;i--){
191         *display_val = i;
192         if(i<=3 && i>0){
193             beep(200);
194         }
195         smart_delay(1000);
196     }
197
198     gpio_put(BUZZER,0);
199     *display_val = 10;
200
201     gpio_put(pedG,0);
202     gpio_put(pedR,1);
203
204     beep(500);
205 }
206
207 //=====
```

```
208 void update_system(){
209     switch(current_state){
210
211         case STATE_CAR1_GREEN:
212             set_car1(0,0,1);
213             set_car2(1,0,0);
214             if(ped1_request || ped2_request)
215                 smart_delay(GREEN_FAST);
216             else
217                 smart_delay(GREEN_NORMAL);
218             current_state = STATE_CAR1_YELLOW;
219             break;
220
221         case STATE_CAR1_YELLOW:
222             set_car1(0,1,0);
223             set_car2(1,0,0);
224             smart_delay(YELLOW_TIME);
225             if(ped1_request)
226                 current_state = STATE_PED1;
227             else if(ped2_request)
228                 current_state = STATE_PED2;
229             else
230                 current_state = STATE_CAR2_GREEN;
231             break;
232
233         case STATE_CAR2_GREEN:
234             set_car1(1,0,0);
235             set_car2(0,0,1);
236             if(ped1_request || ped2_request)
237                 smart_delay(GREEN_FAST);
238             else
239                 smart_delay(GREEN_NORMAL);
240             current_state = STATE_CAR2_YELLOW;
241             break;
242
243         case STATE_CAR2_YELLOW:
244             set_car1(1,0,0);
245             set_car2(0,1,0);
246             smart_delay(YELLOW_TIME);
247             if(ped1_request)
248                 current_state = STATE_PED1;
249             else if(ped2_request)
250                 current_state = STATE_PED2;
251             else
252                 current_state = STATE_CAR1_GREEN;
253             break;
254
255         case STATE_PED1:
```

```
256         ped1_request = false;
257         set_car1(1,0,0); // rojo
258         set_car2(0,0,1); // verde
259         pedestrian_crossing(PEA1R, PEA1G, &disp1_val);
260         current_state = STATE_CAR2_GREEN;
261         break;
262
263     case STATE_PED2:
264         ped2_request = false;
265         set_car1(0,0,1); // verde
266         set_car2(1,0,0); // rojo
267         pedestrian_crossing(PEA2R, PEA2G, &disp2_val);
268         current_state = STATE_CAR1_GREEN;
269         break;
270     }
271 }
272
273 //=====
274 void init_all(){
275     int outputs[] = {
276         CAR1R,CAR1G,CAR1B,
277         CAR2R,CAR2G,CAR2B,
278         PEA1R,PEA1G,
279         PEA2R,PEA2G,
280         BUZZER,
281         SEG_A,SEG_B,SEG_C,
282         SEG_D,SEG_E,SEG_F,SEG_G,
283         COM_DISP1,COM_DISP2
284     };
285
286     for(int i=0;i<20;i++){
287         init_output(outputs[i]);
288     }
289
290     init_input(PUSH1);
291     init_input(PUSH2);
292
293     gpio_put(COM_DISP1,1);
294     gpio_put(COM_DISP2,1);
295
296     gpio_put(PEA1R,1);
297     gpio_put(PEA1G,0);
298
299     gpio_put(PEA2R,1);
300     gpio_put(PEA2G,0);
301
302     gpio_put(BUZZER,0);
303
```



```
304     disp1_val = 10;
305     disp2_val = 10;
306 }
307
308 // =====
309 int main(){
310     stdio_init_all();
311     init_all();
312
313     while(true){
314         mux_tick();
315         check_buttons();
316         update_system();
317     }
318
319     return 0;
320 }
```

Anexo B – Archivo CMakeLists.txt

```
1  # == DO NOT EDIT THE FOLLOWING LINES for the Raspberry Pi Pico
   VS Code Extension to work ==
2  if(WIN32)
3      set(USERHOME $ENV{USERPROFILE})
4  else()
5      set(USERHOME $ENV{HOME})
6  endif()
7  set(sdkVersion 2.2.0)
8  set(toolchainVersion 14_2_Rel1)
9  set(picotoolVersion 2.2.0-a4)
10 set(picoVscode ${USERHOME}/.pico-sdk/cmake/pico-vscode.cmake)
11 if (EXISTS ${picoVscode})
12     include(${picoVscode})
13 endif()
14 #
   =====
15
16 cmake_minimum_required(VERSION 3.13)
17
18 set(PICO_BOARD pico_w)
19
20 include(pico_sdk_import.cmake)
21
22 project(Practica_14)
23
```

```

24 pico_sdk_init()
25
26 add_executable(Practica14
27     Practica14.c
28 )
29
30 pico_enable_stdio_usb(Practica14 1)
31 pico_enable_stdio_uart(Practica14 0)
32
33 target_link_libraries(Practica14
34     pico_stdlib
35     hardware_timer
36 )
37
38 pico_add_extra_outputs(Practica14)

```

Anexo C – Tabla de Patrones del Display de 7 Segmentos

Cuadro 4: Códigos de patrones para display de 7 segmentos de cátodo común

Dígito	Código Hex	Segmentos activos
0	0x3F	A B C D E F
1	0x06	B C
2	0x5B	A B D E G
3	0x4F	A B C D G
4	0x66	B C F G
5	0x6D	A C D F G
6	0x7D	A C D E F G
7	0x07	A B C
8	0x7F	A B C D E F G
9	0x6F	A B C D F G
Apagado	0x00	(ninguno)

Evidencia Fotográfica

En esta sección se presentan fotografías del prototipo físico construido en proto-board, mostrando el estado del sistema durante su operación normal y durante un cruce peatonal activo.

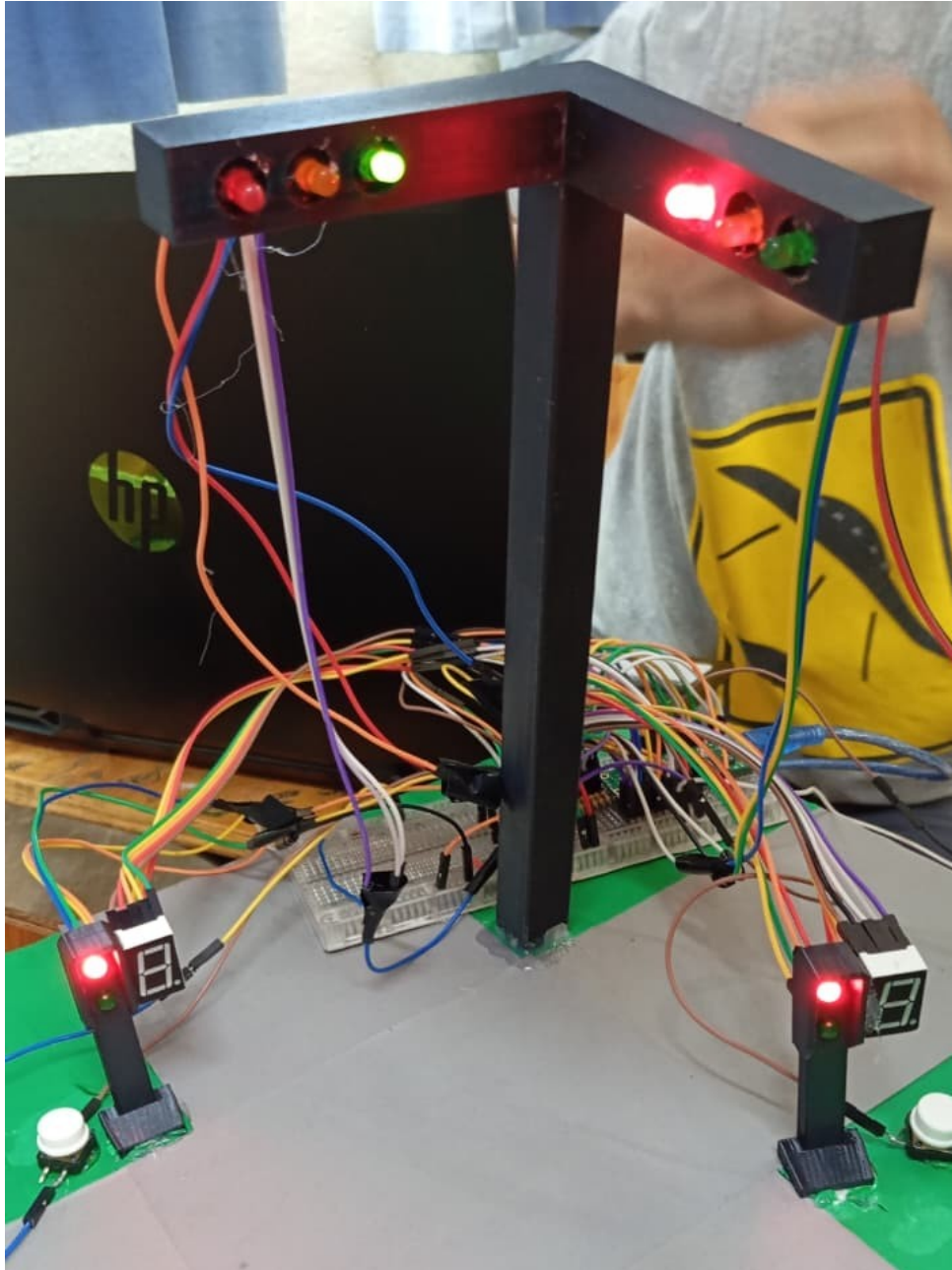


Figura 3: Vista general del prototipo del sistema de cruce peatonal armado en protoboard. Se aprecia la Raspberry Pi Pico W al centro, los semáforos vehiculares y peatonales con sus LEDs, los displays de 7 segmentos y los botones de solicitud de cruce.

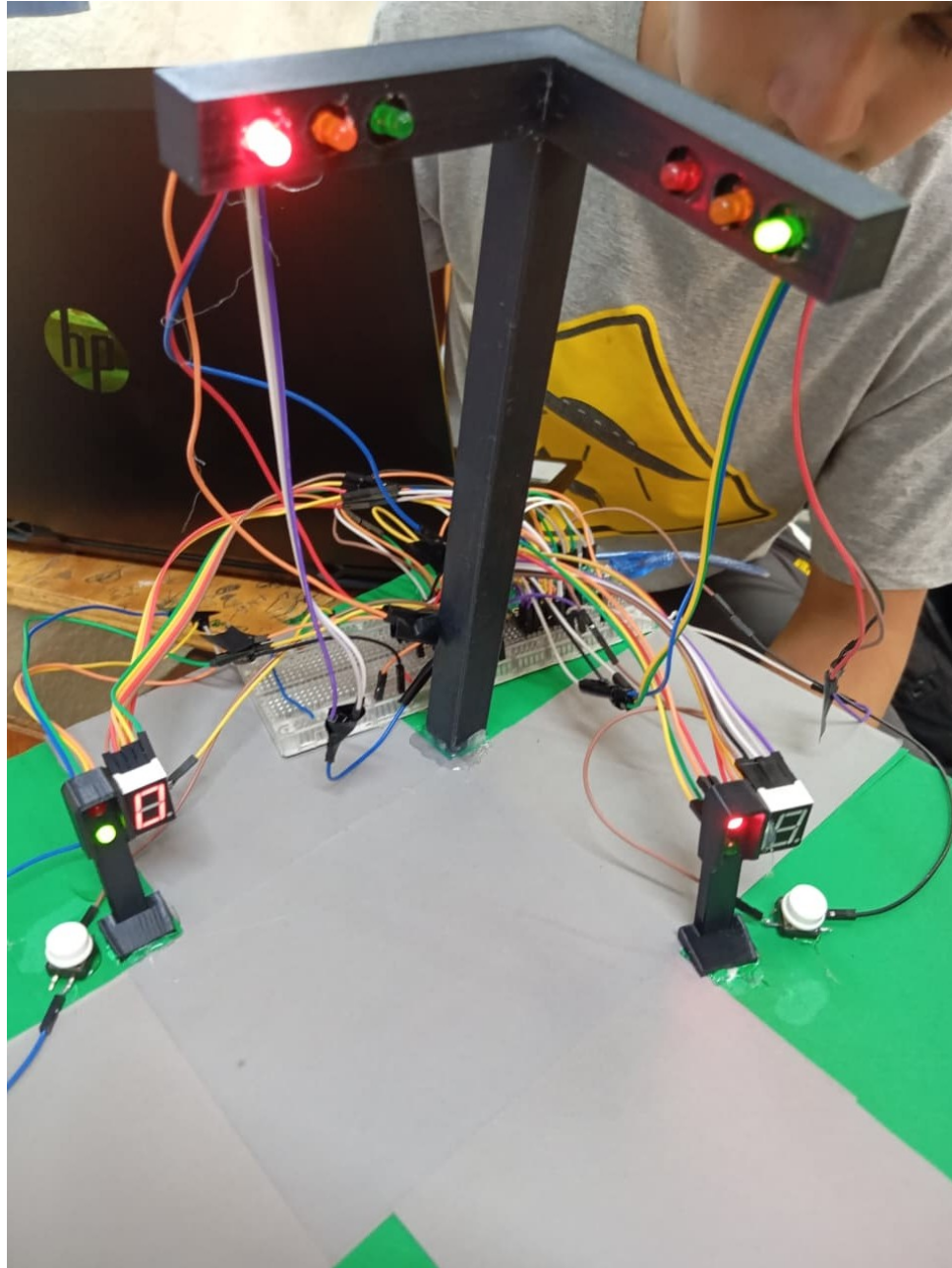


Figura 4: Sistema durante un cruce peatonal activo: semáforo peatonal en verde con el display mostrando la cuenta regresiva, y semáforo vehicular correspondiente en rojo.

Referencias

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico C/C++ SDK* (v2.2). <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>, 2024.
- [2] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*. <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>, 2024.

- [3] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.
- [4] E. Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [5] Texas Instruments, *Seven-Segment Display Multiplexing Techniques*. Application Report SLOA031, 2002.